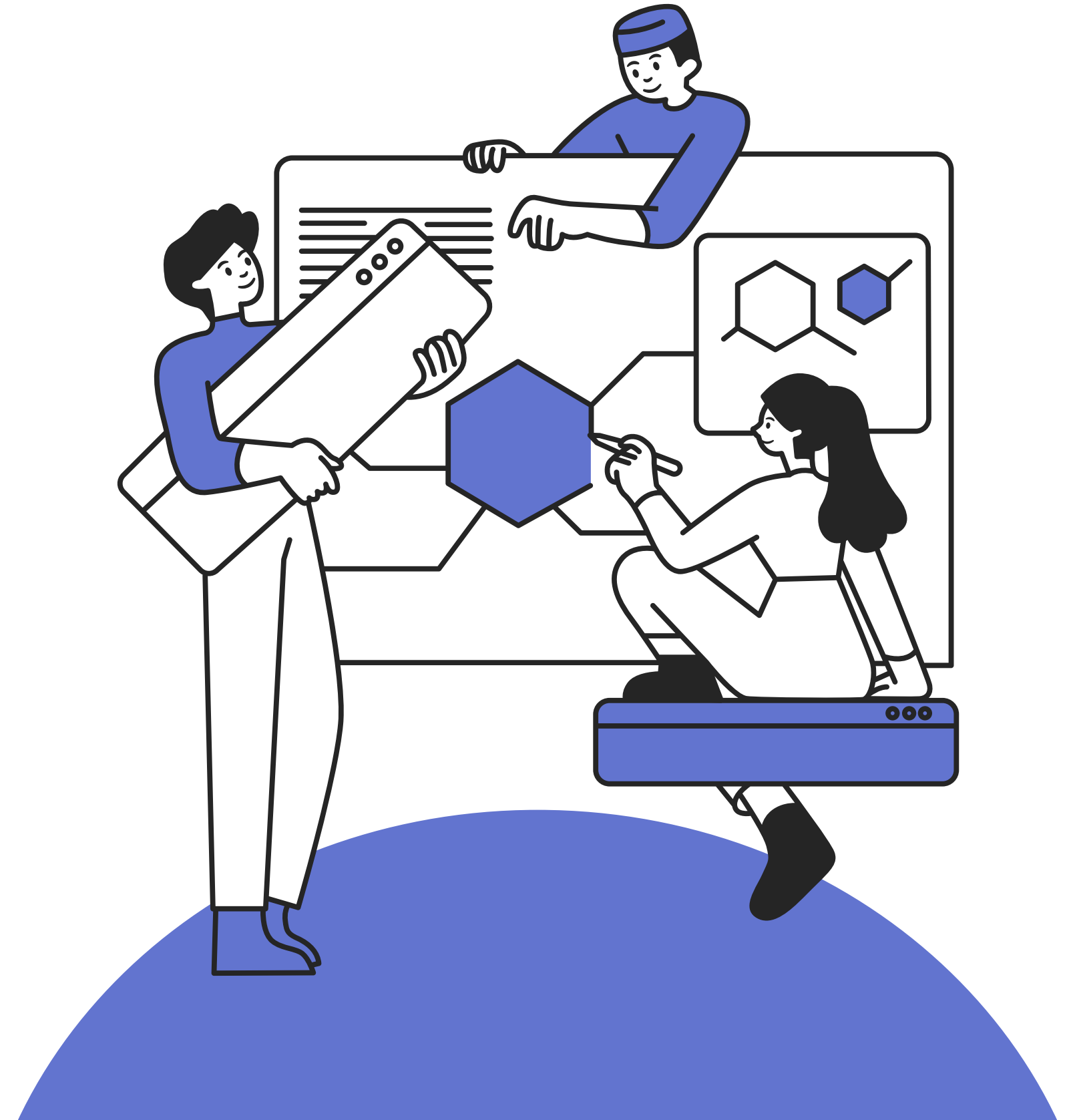


탐욕 알고리즘

이예린T



CONTENTS

- 1 탐욕 알고리즘
- 2 탐욕 알고리즘 대표 문제
- 3 모듈별 탐구
- 4 탐욕 알고리즘 정리



01

탐욕 알고리즘

Greedy algorithm

- 탐욕스럽게 가까운 것만 바라보고 근시안적으로 최적해를 구한다. 바로 눈앞의 이익만을 좇는다.
- 글로벌 최적을 찾기 위해 각 단계에서 로컬 최적의 선택을 하는 휴리스틱 문제 해결 알고리즘이다.
- 탐욕 알고리즘은 효율적이기는 하지만 반드시 최적해를 구해준다는 보장은 할 수 없다.
- 대부분의 경우 뛰어난 결과를 도출하지 못하지만, 드물게 최적해를 보장하는 경우도 있다.(문제에 따라 다름)
- 최적해를 찾을 수 있으면 그것이 목표가 되고, 찾기 어려운 경우 주어진 시간 내에 그런대로 괜찮은 해를 찾는 것을 목표로 삼는다.
- 합리적인 시간 내에 최적에 가까운 답을 찾을 수 있다.

01

탐욕 알고리즘

Greedy algorithm

- 탐욕 알고리즘이 적절한 문제들은 탐욕 선택 속성을 갖고 있는 최적 부분 구조인 문제들이다.
 - 탐욕 선택 속성: 앞의 선택이 이후 선택에 영향을 주지 않는 것. 선택을 다시 고려하지 않는다.
 - 최적 부분 구조: 문제의 최적 해결 방법이 부분 문제에 대한 최적 해결 방법으로 구성되는 경우
- 탐욕 알고리즘의 동작 과정
 - 1) 해 선택: 현재 상태에서 부분 문제의 최적해를 구한 후 이를 부분해 집합에 추가한다.
 - 2) 실행 가능성 검사: 새로운 부분해 집합이 실행 가능한 것인지 확인한다. 다시 말해, 문제의 제약 조건을 위반하지 않는지를 검사한다.
 - 3) 새로운 부분해 집합이 문제의 해가 되는지 확인한다. 아직 전체 문제의 해가 완성되지 않았다면 단계 1)의 해 선택부터 다시 시작한다.
- 대표적인 탐욕 알고리즘 문제로 거스름돈 문제, (분할 가능한) Knapsack problem(배낭 문제) 등이 있다.

01

거스름돈 문제

지폐와 동전을 어떻게 구성하여 줄 것인가?



- 원하던 대학에 합격한 세종이는 편의점 아르바이트를 하고 있다. 세종이는 사려 깊은 직원이라서, 거스름돈을 줄 때 어떻게 해야 편의점 고객이 가장 덜 불편할지 고민했다. 그 결과, 지폐와 동전의 개수를 최대한 적게 줘야겠다고 생각했다.
- **그렇다면 고객에게 N원을 줘야 할 때, 어떤 지폐와 동전을 몇 개씩 주면 될까?**
- 단, 화폐는 한국 돈을 기준으로 한다. (50000, 10000, 5000, 1000, 500, 100, 50, 10, 5, 1원)

01

거스름돈 문제

간쵸 거스름돈 주기



- 1550원짜리 간쵸 하나를 산 고객이 10000원짜리 지폐 한 장을 냈다. 거스름돈을 어떻게 줄 것인가?
 - 모든 지폐, 동전 종류를 각각 무한히 가지고 있다고 가정할 때
 - 1000원짜리 10장, 50원짜리 8개, 10원짜리 10개 있다고 가정할 때
- 탐욕 알고리즘은 항상 최소 개수의 지폐와 동전을 보장하는가?

01

거스름돈 문제

400원짜리 동전



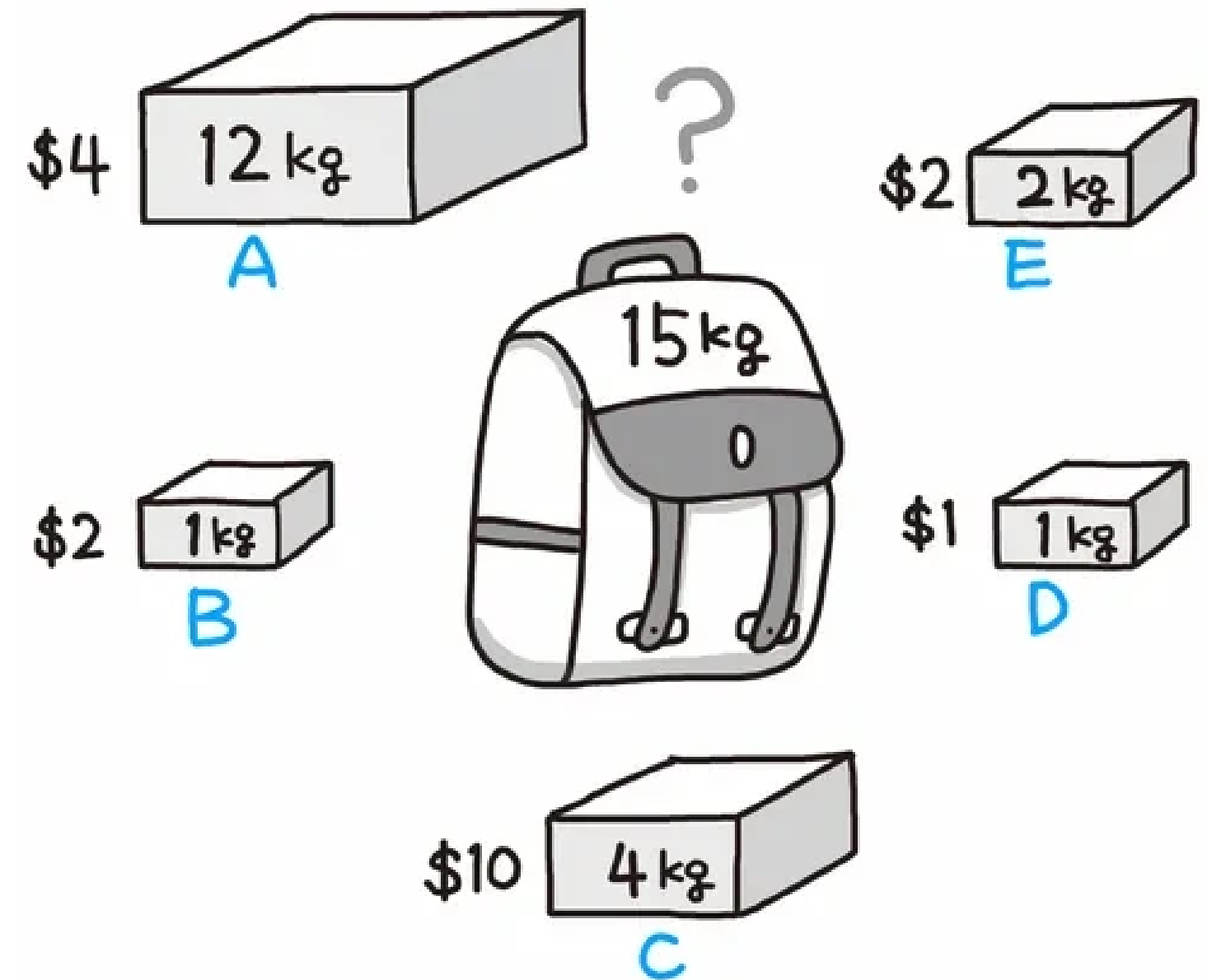
- 50000, 10000, 5000, 1000, 500, 100, 50, 10, 5, 1원짜리
- 한국은행에서 400원짜리 동전을 발행한다면?
- 손님에게 800원 주는 방법은?
 - 탐욕 알고리즘을 적용해 보자.
- 큰 단위가 작은 단위의 배수일 때만 탐욕 알고리즘이 최적의 해를 구한다.

02

배낭 문제

Knapsack problem

- 세종이가 인생이 안 풀려서 도둑질을 하는 상상을 하게 됐다.
- 학교 가는 척 배낭을 메고, 빈 집을 털어 배낭에 최대한 높은 값어치를 담아 유유히 빠져나올 계획이다.
- 세종이 **배낭이 담을 수 있는 최대 무게가 15kg이고, 빈 집에 물건들이 다음과 같이 있을 때, 세종이는 배낭에 무엇을 얼마나 넣어야 할까?** (단, 물건은 자르거나 쪼개어 담을 수 있고, 무게 당 가치는 유지된다.)

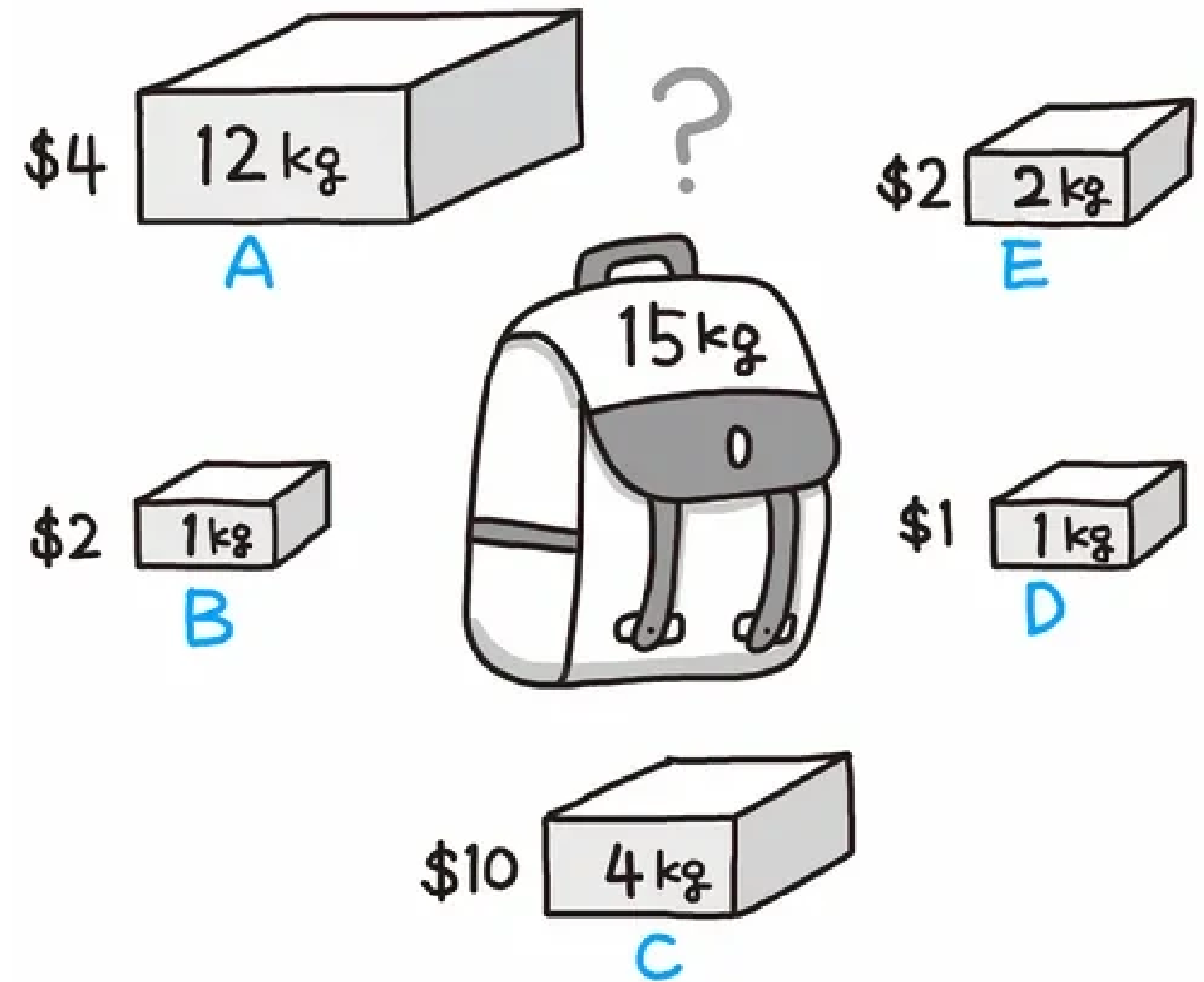


02

배낭 문제 물건을 쪼개?



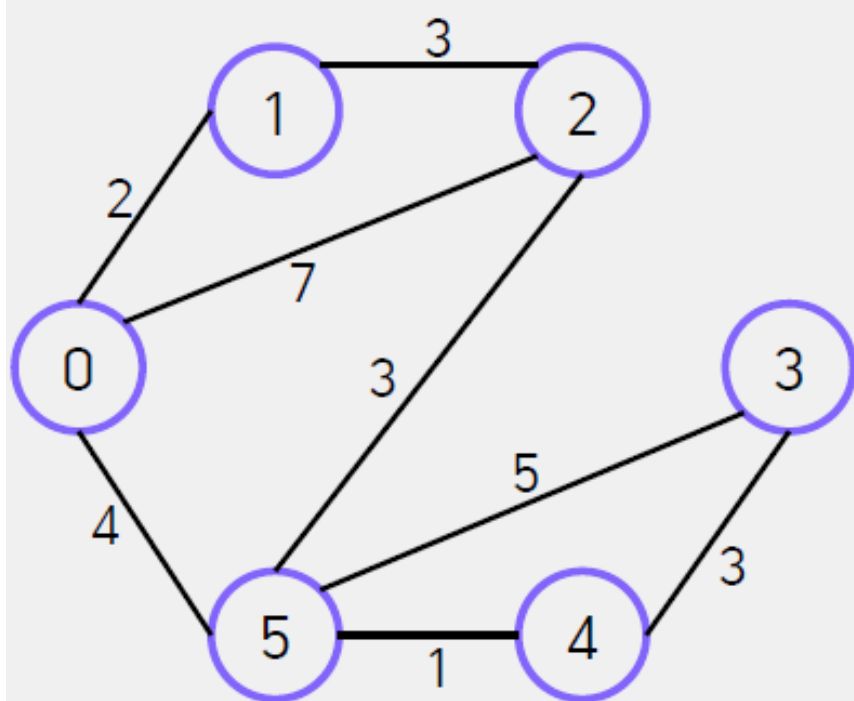
- 분할 가능 배낭 문제는 탐욕 알고리즘을 적용해 해결할 수 있다.
- 단가가 가장 높은 짐부터 차례대로 채워나간다.
- 그런데, 물건을 자르거나 쪼갤 수 없다면? (<< 0-1 배낭 문제)
- 탐욕 알고리즘을 적용할 수 있을까?



03

Dijkstra 다시 보기

매번 '가장 작은 값'을 선택함.



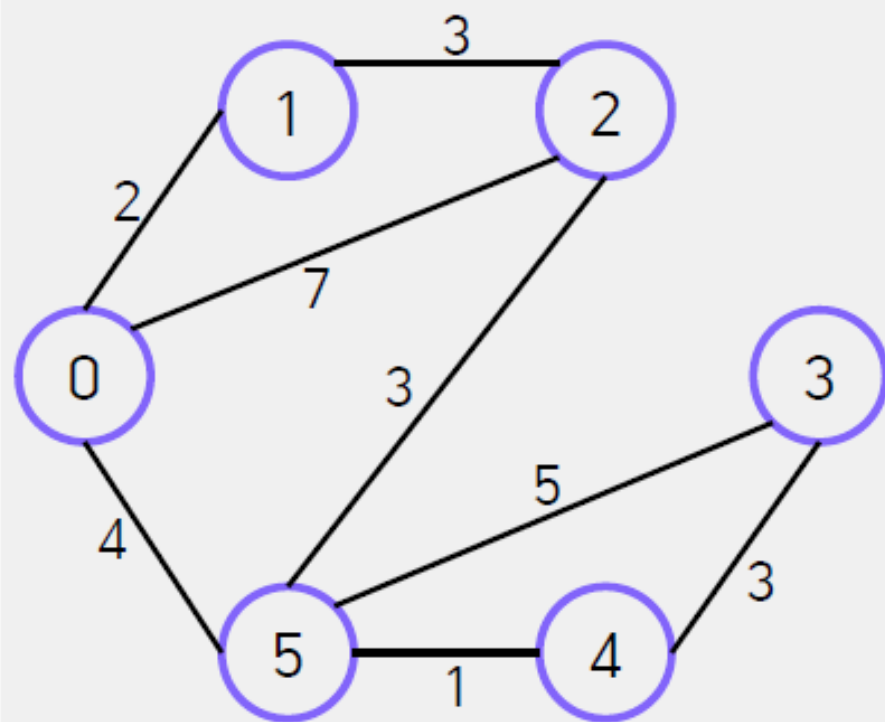
	V					
	0	1	2	3	4	5
시작	0	2	7	inf	inf	4
[1]	0	2	5	inf	inf	4

- 윗줄에서 V 체크 없는 노드 중 가장 작은 경로를 가진 것 선택(비용 2인 1 노드 선택)
- V 체크한 노드는 윗줄 값 그대로
- 나머지 노드는 새로운 경로와 비교
- 선택한 노드(1)에서 연결된 노드(2)에 대해 새로운 경로 비용 검사
- 노드1까지의 비용(2) + 노드1에서 노드2까지의 비용(3) VS 기존에 알던 노드2까지의 비용(4)

03

Dijkstra 다시 보기

매번 '가장 작은 값'을 선택함.



	V	V	V		V	V
	0	1	2	3	4	5
시작	0	2	7	inf	inf	4
[1]	0	2	5	inf	inf	4
[5]	0	2	5	9	5	4
[2]	0	2	5	9	5	4
[4]	0	2	5	8	5	4

- 윗줄에서 V 없는 것 중 가장 작은 경로를 가진 노드 선택(비용 5인 4)
- V 체크한 노드는 윗줄 값 그대로
- 나머지 노드는 새로운 경로와 비교
- 선택한 노드(4)에서 연결된 노드(3)에 대해 새로운 경로 비용 검사
- 노드4까지의 비용(5) + 노드4에서 노드3까지의 비용(3) VS 기존에 알던 노드3까지의 비용(9)

03

Dijkstra 다시 보기

매번 ‘가장 작은 값’을 선택함.

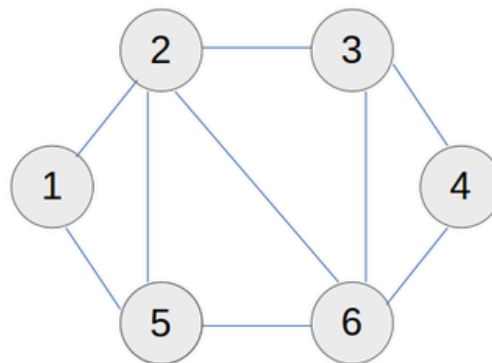
- 탐욕 알고리즘에 해당
 - 매 단계에서 ‘현재까지 가장 짧은 거리’를 갖는 노드를 선택한다.
 - 한 번 확정된 노드는 다시 돌아보지 않는다.
- 탐욕 알고리즘 관점에서 다익스트라 이해하기
- 1) 해 선택: 미방문 노드 중 출발지로부터 거리가 가장 짧은 노드를 선택하고, 경로 확정 집합에 추가한다.
 - 미방문 노드 중 dist 최솟값 노드 u 선택 $\rightarrow S$ 에 추가
- 2) 실행 가능성 검사: 선택한 노드의 인접 노드에 대해, 현재 경로가 최단 거리인지 확인하고, 더 짧은 경로를 찾았다면 정보를 업데이트한다.
 - u 의 인접 노드 v 에 대해 $\text{dist}[v]$ 갱신
- 3) 해 완성 검사: 미방문 노드가 없으면 종료, 있으면 1)로 반복

04

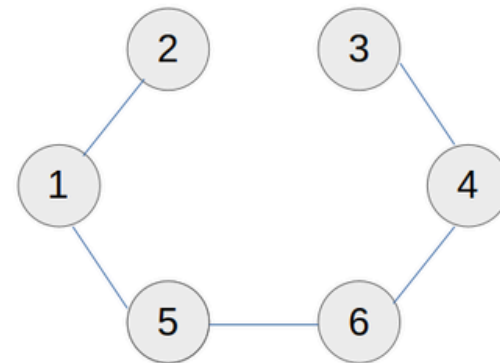
Kruskal 알고리즘

최소 신장 트리

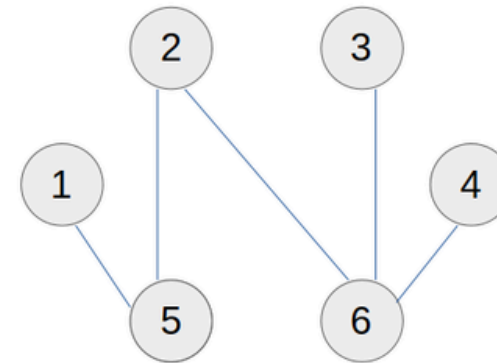
- 최소 신장 트리(Minimum Spanning Tree): 여러 간선 중 가중치의 합이 최소가 되는 간선만 남겨 그래프의 모든 정점을 사이클 없이 연결한 트리
 - 신장 트리(Spanning Tree): 그래프의 모든 노드를 포함하면서 사이클이 없는 부분 그래프 (간선 수 = 노드 수 - 1)
 - 최소 신장 트리: 신장 트리 중 간선 가중치의 합이 최소인 것



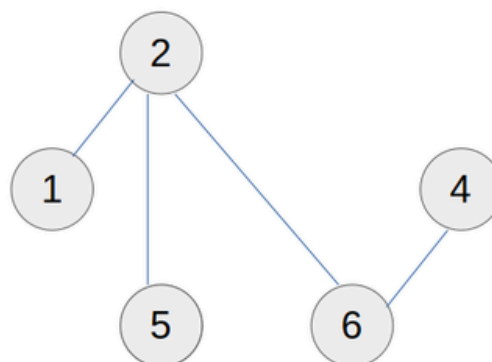
원래의 그래프



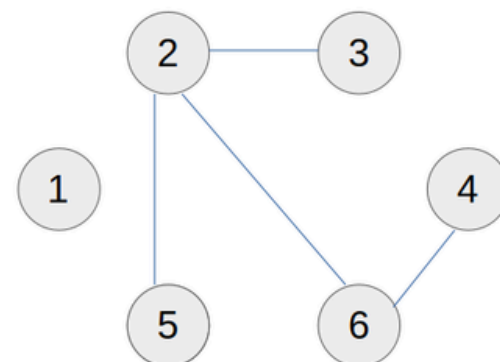
신장 트리의 예시 1



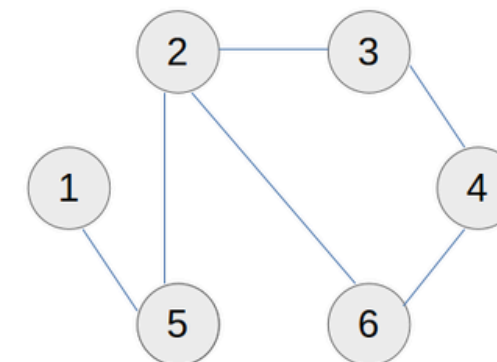
신장 트리의 예시 2



신장 트리가 아님: 모든 노드를 포함하지 않음

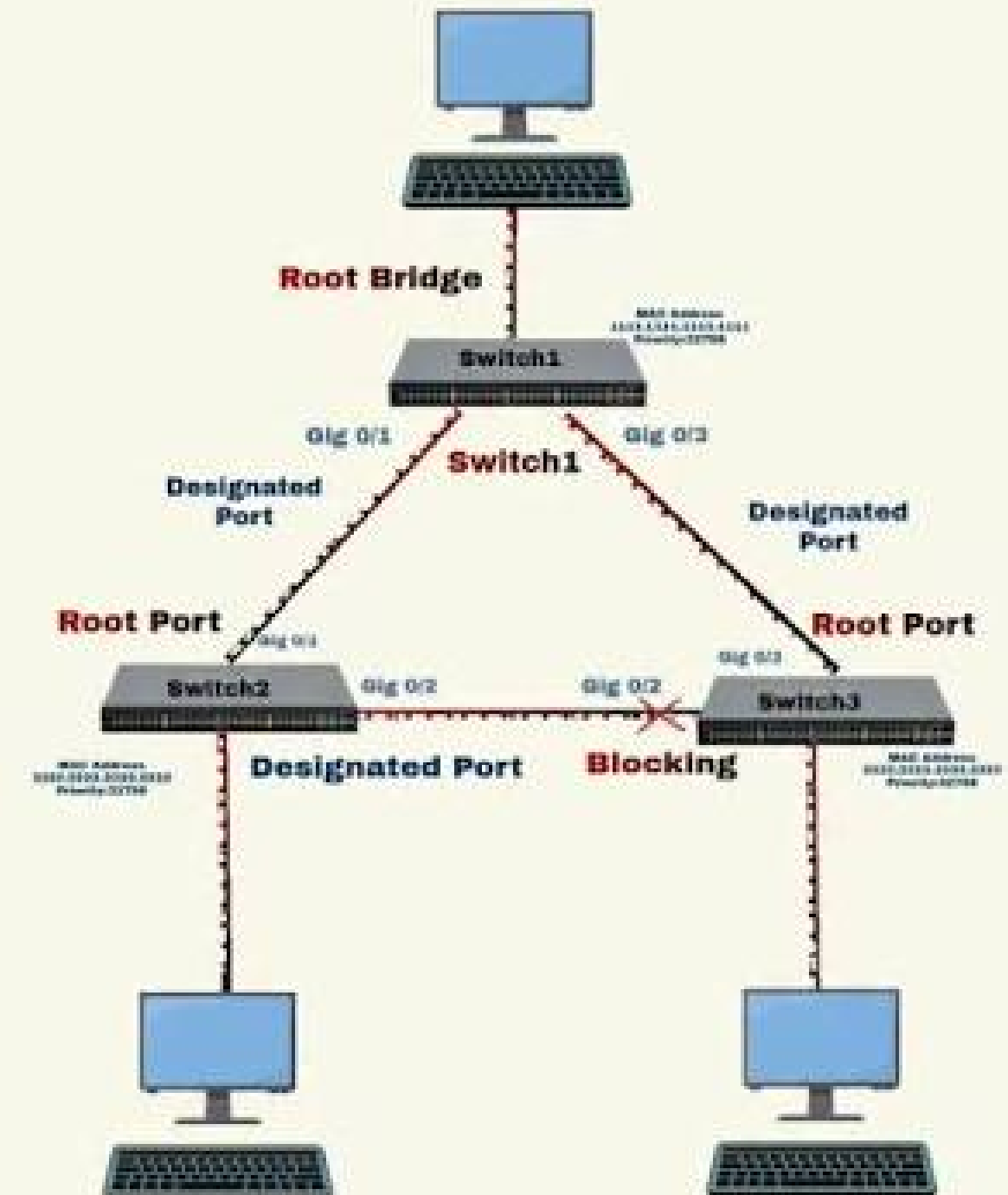


신장 트리가 아님: 연결 그래프여야 함



신장 트리가 아님: 사이클이 없어야 함

Spanning Tree Protocol (STP)

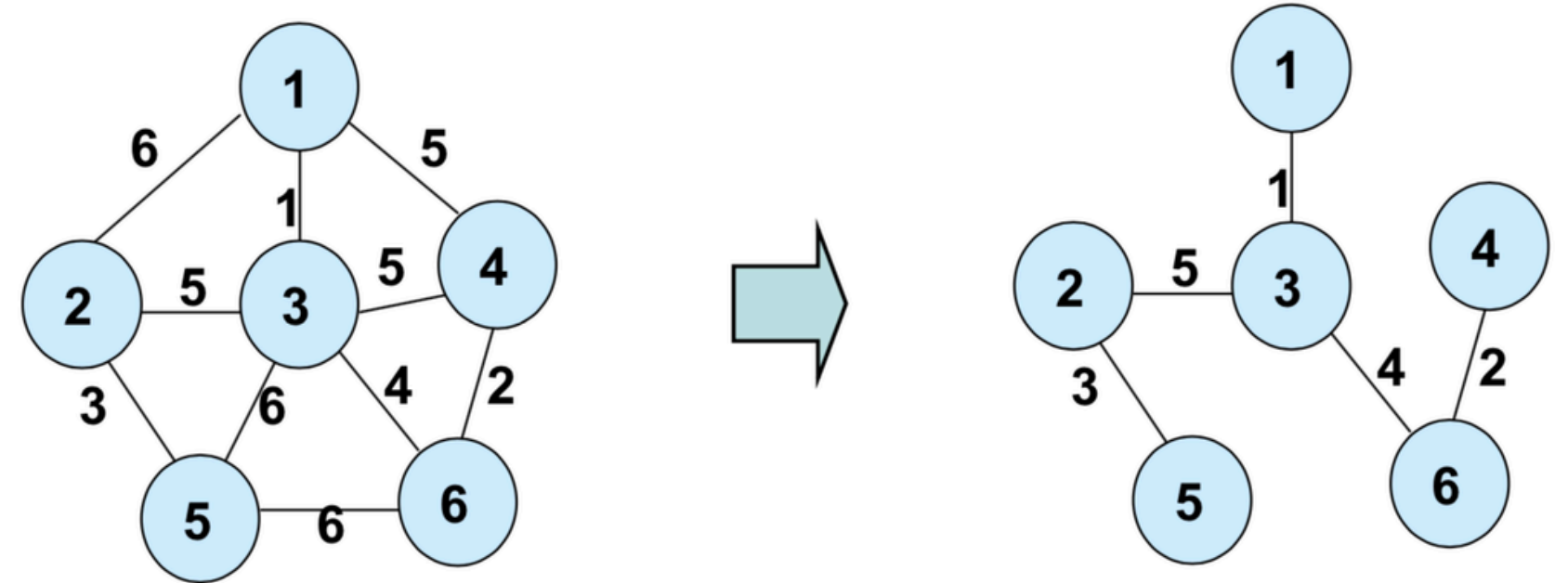


04

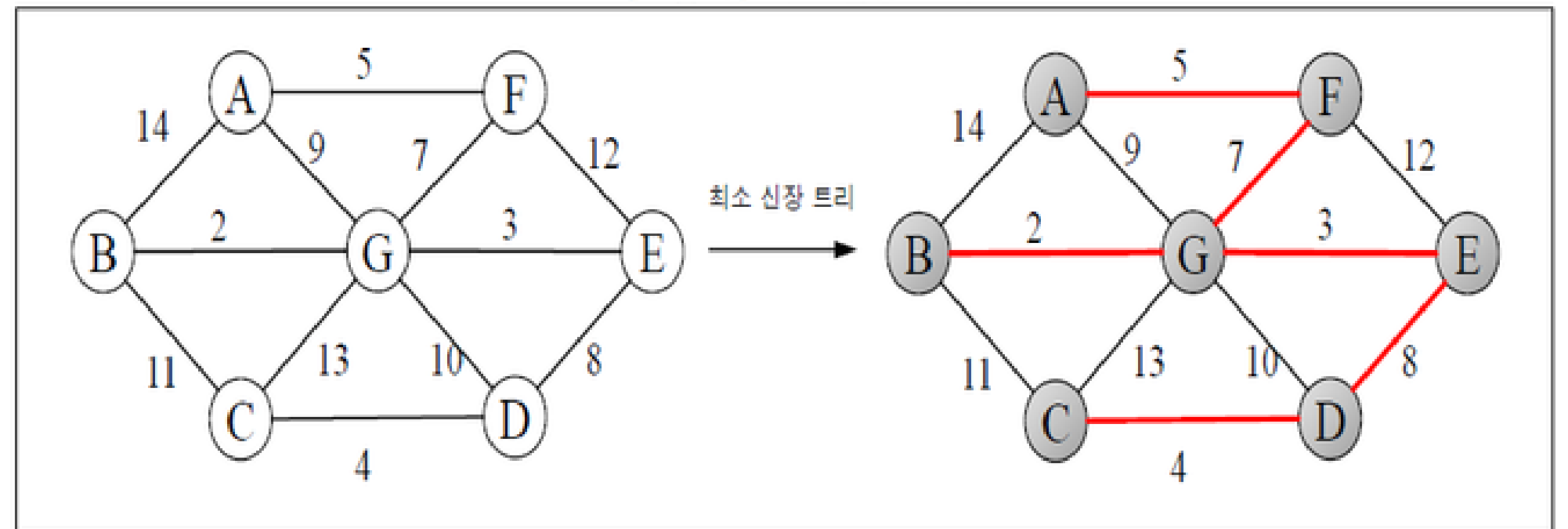
Kruskal 알고리즘

최소 신장 트리

- 최소 신장 트리가 필요한 문제
 - 최소한의 비용으로 모든 도시를 연결하는 도로 건설하기
 - 새로 건설할 건물의 배관을 최소 비용으로 구축하기
 - 책상이 복잡하게 놓인 사무실에 전기선 연결하기
- 대표적인 최소 신장 트리 알고리즘: Prim, Kruskal



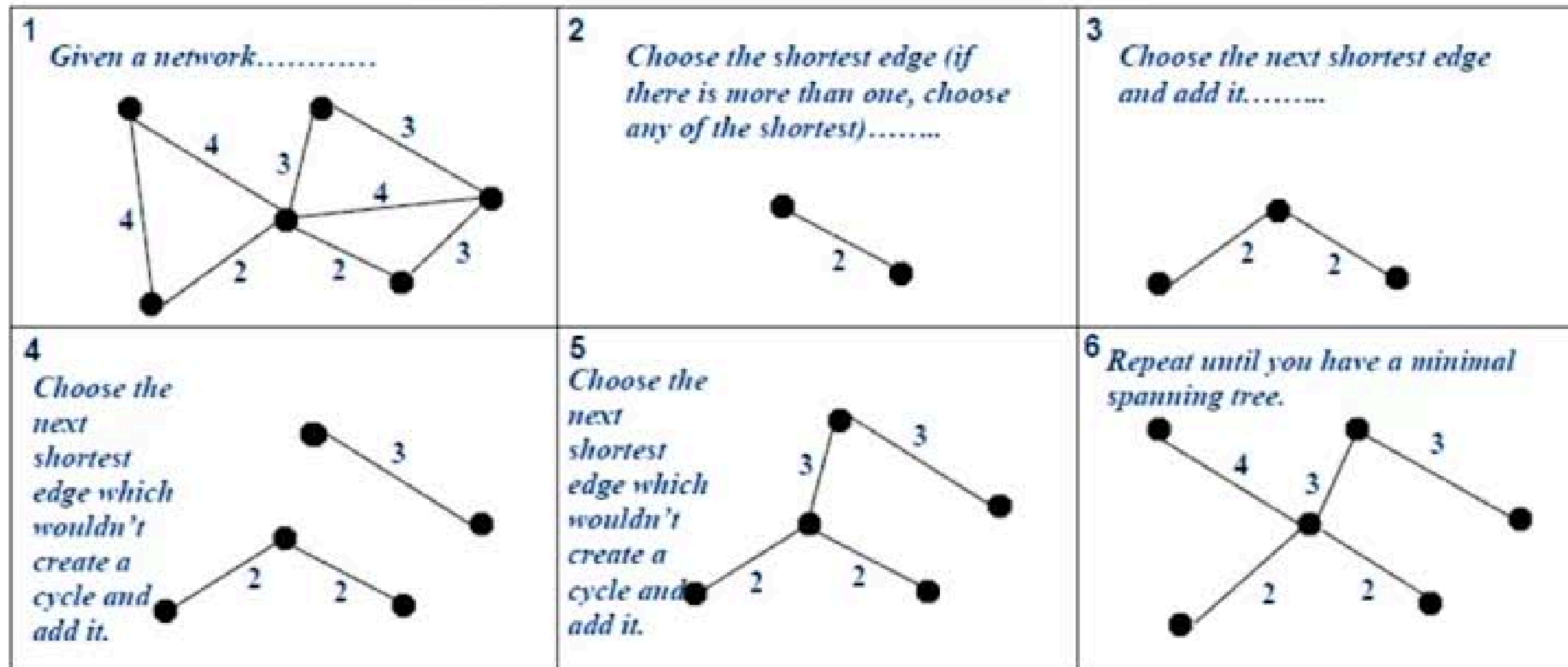
[그림-1] 최소 신장 트리



04

Kruskal 알고리즘

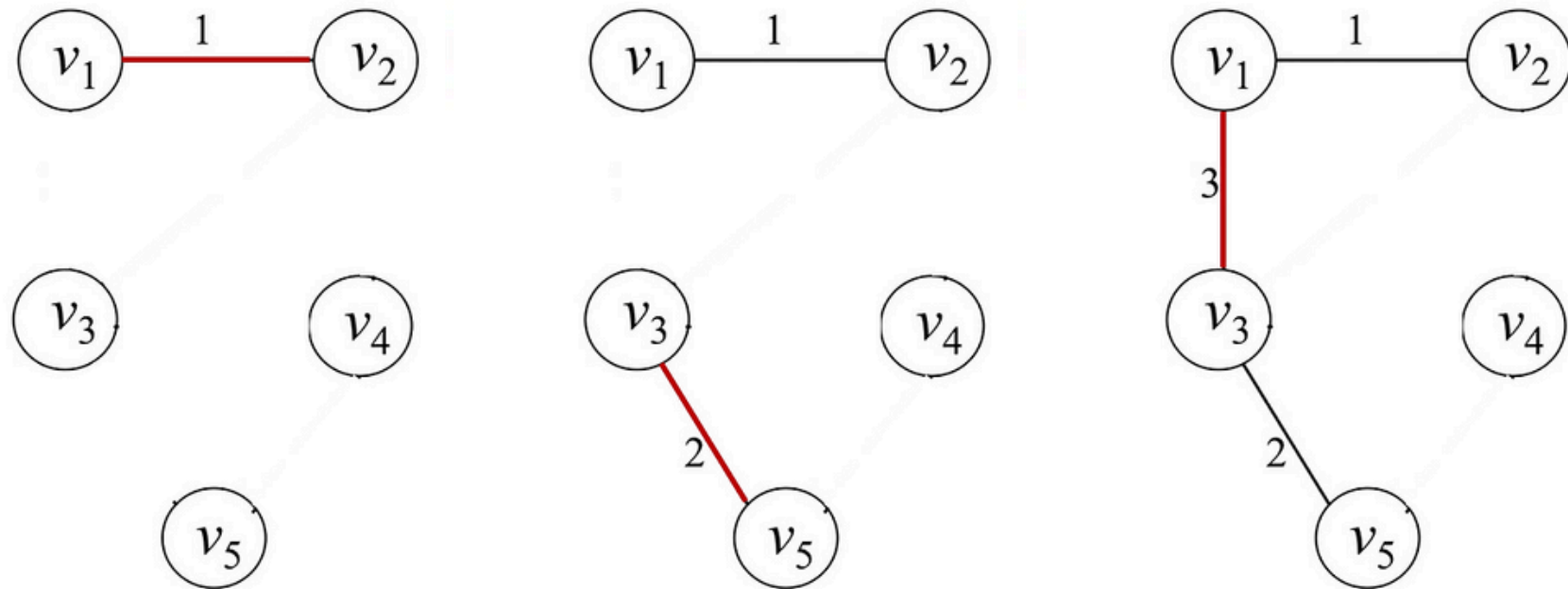
- Kruskal 알고리즘
 - 1) 그래프 내의 모든 간선을 가중치 오름차순으로 정렬
 - 2) 간선 목록을 차례로 순회하며 간선을 최소 신장 트리에 추가한다. 단, 이때 추가된 간선으로 인해 최소 신장 트리 내에 **사이클이 형성되면 안 된다.** (-> 사이클 형성되는 간선은 무시하고 다음 간선 채택)



04

Kruskal 알고리즘

- 사이클이 있는지 없는지 어떻게 알아?
 - 집합을 활용해 보자.
 - 각 정점별로 각각의 분리 집합을 만들고, 간선으로 연결된 정점들에 대해서는 합집합을 수행한다.



(v_1, v_2) 1
 (v_3, v_5) 2
 (v_1, v_3) 3
 (v_2, v_3) 3
 (v_3, v_4) 4
 (v_4, v_5) 5
 (v_2, v_4) 6

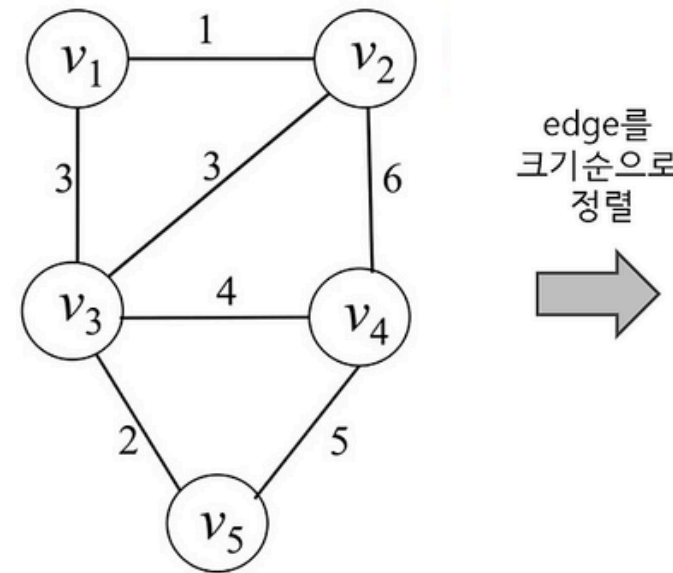
$\{v_1, v_2\}, \{v_3\},$
 $\{v_4\}, \{v_5\}$

(v_1, v_2) 1
 (v_3, v_5) 2
 (v_1, v_3) 3
 (v_2, v_3) 3
 (v_3, v_4) 4
 (v_4, v_5) 5
 (v_2, v_4) 6

$\{v_1, v_2\},$
 $\{v_4\}, \{v_3, v_5\}$

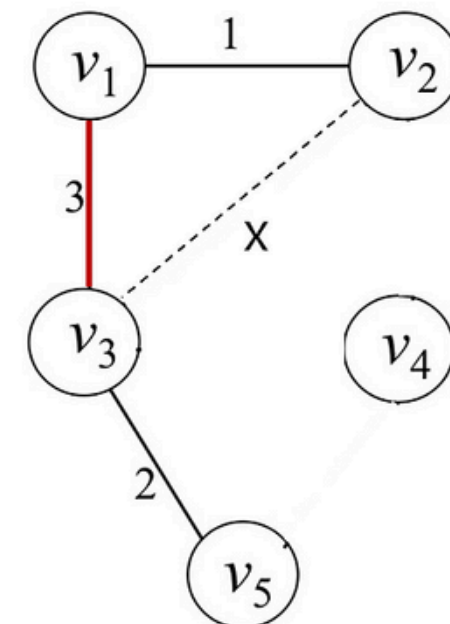
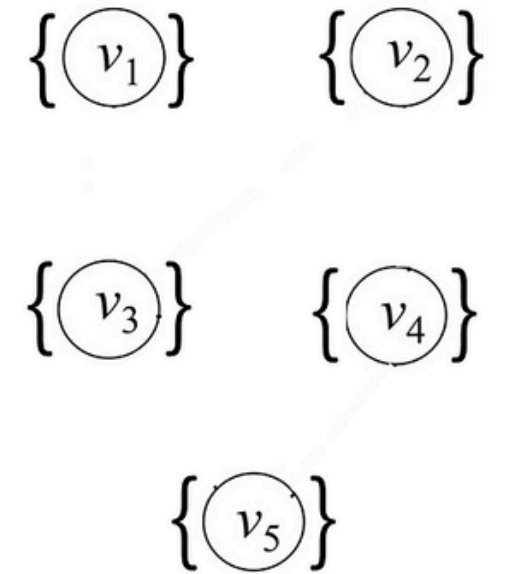
(v_1, v_2) 1
 (v_3, v_5) 2
 (v_1, v_3) 3
 (v_2, v_3) 3
 (v_3, v_4) 4
 (v_4, v_5) 5
 (v_2, v_4) 6

$\{v_1, v_2, v_3, v_5\},$
 $\{v_4\}$



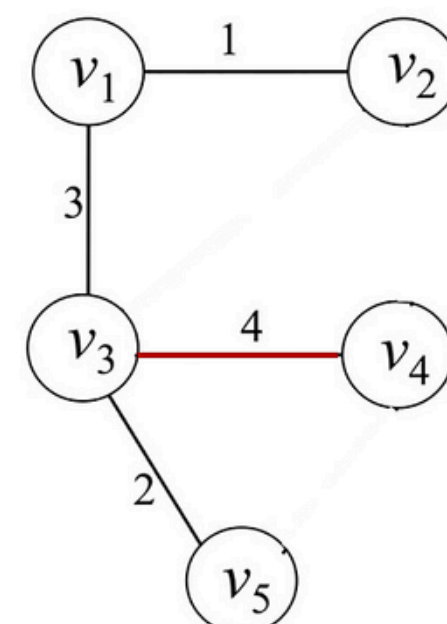
edge를
크기순으로
정렬

(v_1, v_2) 1
 (v_3, v_5) 2
 (v_1, v_3) 3
 (v_2, v_3) 3
 (v_3, v_4) 4
 (v_4, v_5) 5
 (v_2, v_4) 6



(v_1, v_2) 1
 (v_3, v_5) 2
 (v_1, v_3) 3
 (v_2, v_3) 3
 (v_3, v_4) 4
 (v_4, v_5) 5
 (v_2, v_4) 6

X, 서로 같은 집합



(v_1, v_2) 1
 (v_3, v_5) 2
 (v_1, v_3) 3
 (v_2, v_3) 3
 (v_3, v_4) 4
 (v_4, v_5) 5
 (v_2, v_4) 6

$\{v_1, v_2, v_3, v_4, v_5\}$

모든 정점들이 연결되었으므로
더 이상 고를 필요 X

04

Kruskal 알고리즘

- 탐욕 알고리즘 관점에서 Kruskal 알고리즘 이해하기
 - 1) 해 선택: 미선택 간선 중 가중치 최소 간선 선택
 - 2) 실행 가능성 검사: 선택한 간선이 사이클을 만드는지 확인
 - 3) 해 완성 검사: 간선 수 = 노드 수 - 1 이면 종료, 아니면 1)로 가서 반복

01

거스름돈 문제

- 1단계: 화폐 단위 입력
 - 사용자로부터 사용할 지폐/동전의 종류를 입력받는다.
 - 입력받은 화폐 단위들이 서로 배수 관계를 이루는지 검사한다.
 - 배수 관계가 아니면 탐욕 알고리즘으로 최적해를 보장할 수 없으므로 프로그램을 종료한다.
- 2단계: 화폐 보유 수량 입력
 - 각 화폐가 무한히 있는지, 유한히 있는지 선택받는다.
 - 유한한 경우 각 화폐 단위별 보유 개수를 입력받는다.
- 3단계: 거래 정보 입력
 - 손님이 지불한 금액과 실제 구매 금액을 입력받는다.
 - 두 금액의 차이를 계산하여 거슬러줄 금액을 구한다.
- 4단계: 거스름돈 계산 및 출력
 - 탐욕 알고리즘으로 거스름돈을 최소 개수의 화폐로 반환하는 방법을 계산한다.
 - 각 화폐 단위별 사용 개수를 출력한다.
 - 보유 수량이 부족하여 거스름돈을 줄 수 없는 경우 불가 메시지를 출력한다.

02

도시 MST 만들고, 분할하기

- N개의 마을과 M개의 길이 있다. 각 길에는 유지 비용이 있는데, 세종이는 두 가지 버전의 도시 계획을 하고자 한다.
- 1안: 모든 마을을 최소 비용으로 연결하기
 - 입력
 - 마을 수 N, 길의 수 M
 - M개의 길 정보: (마을1, 마을2, 비용)
 - 출력
 - 간선을 추가할 때마다 집합 상태 변화 출력
 - MST에 포함된 간선 목록
 - MST 전체 비용 합
- 2안: 마을을 두 구역으로 나눴을 때 각 구역의 유지 비용 합이 최소가 되도록 연결하기
 - 조건
 - 각 구역 내 마을들은 서로 연결되어 있어야 한다.
 - 두 구역 사이의 길은 모두 제거한다.
 - 두 구역의 유지 비용 합이 최소가 되어야 한다.
 - 출력
 - 제거한 간선 정보
 - 각 구역에 속한 마을 목록과 유지 비용 합
 - 두 구역의 유지 비용 합 (최솟값)

03

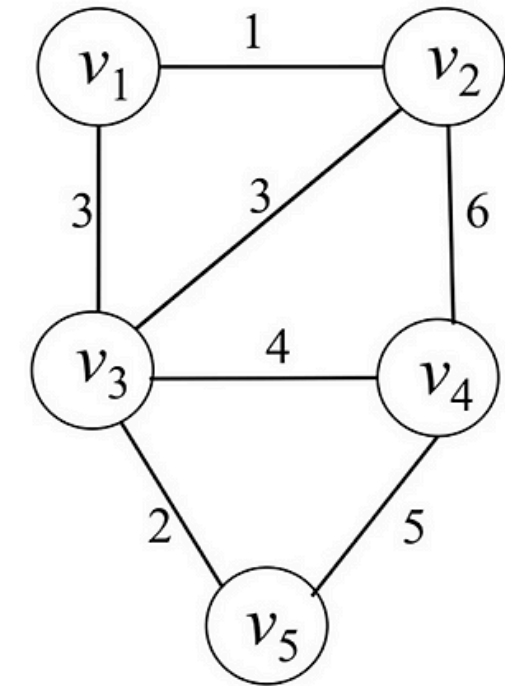
Prim 알고리즘

- 대표적인 최소 신장 트리 알고리즘으로 Kruskal 외에 Prim 알고리즘도 있다.

1) Prim 알고리즘을 소개하고, 동작 과정을 제시한 후, 아래 그래프에 적용해 단계별로 설명하시오.

2) Prim 알고리즘을 우선순위 큐를 활용하여 구현하시오.

3) Prim 알고리즘과 Kruskal 알고리즘을 비교하시오.



04

허프만 코딩

- 문자열을 입력받아 허프만 코딩으로 압축하고, 압축된 결과를 다시 원래 문자열로 복원하는 프로그램을 구현하시오.
 - 입력받은 문자열을 분석하여 허프만 트리를 구성하고 각 문자의 코드를 출력한다.
 - 구성한 허프만 코드를 활용하여 문자열을 압축하고 압축률을 출력한다.
 - 압축된 비트열과 허프만 트리를 활용하여 원래 문자열로 복원한다.
- 허프만 트리 구축 과정을 탐욕 알고리즘 관점에서 해석해 [해 선택 - 실행 가능성 검사 - 해 검사] 각 단계에 해당하는 작업을 서술하시오.

05

큰 수 만들기

- 어떤 숫자에서 k개의 수를 제거했을 때 얻을 수 있는 가장 큰 숫자를 구하려 한다.
 - 예를 들어, 숫자 1924에서 수 두 개를 제거하면 [19, 12, 14, 92, 94, 24] 를 만들 수 있다. 이 중 가장 큰 숫자는 94다.
 - 문자열 형식으로 숫자 number와 제거할 수의 개수 k를 입력받는다. number에서 k개의 수를 제거했을 때 만들 수 있는 수 중 가장 큰 숫자를 문자열 형태로 return하는 함수를 완성하시오.
-
- 단, 스택을 활용하고, 각 단계에서 스택 상태를 출력하도록 구현하시오.
 - 또, 자신이 제시한 풀이가 왜 탐욕 알고리즘이고, 왜 항상 최적해를 보장하는지 서술하시오.

01

탐욕 알고리즘

정리해 보기

- 탐욕 알고리즘: 매 순간 최선의 선택을 하며, 한 번 선택한 건 반복하지 않는다.

→ 구현이 단순하고, **속도가 빠르다**(!!!)

→ 속도가 빠르다는 건 매우 강력한 장점.. 그래서 여기저기 쓰고 싶으나, 언제나 최적해가 보장되지 않는다.

→ 최적해가 보장되는 건, 특정 조건을 만족하는 문제에서만

- (1) 탐욕 선택 속성: 현재의 선택이 이후 선택에 영향을 주지 않아야 함. (5000원짜리 주고 잔액은 잔액대로만 생각한다.)
- (2) 최적 부분 구조: 부분 문제의 최적해가 전체 최적해를 구성해야 함. (5000원짜리 주기로 결정했으면, 거스름돈에 포함됨.)

02

Prim 알고리즘

이것도 탐욕 알고리즘

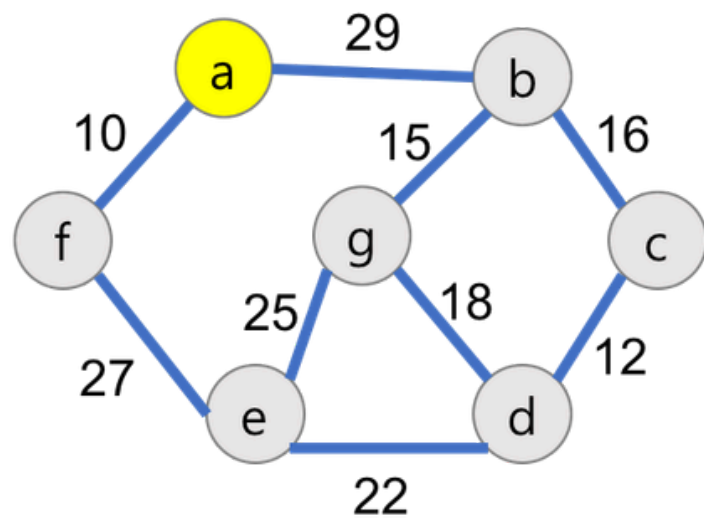
- Prim 알고리즘: 그래프에서 최소 신장 트리를 만들어내는 알고리즘 중 하나
 - 1) 그래프에서 임의의 정점을 시작 정점으로 선택하여 최소 신장 트리의 뿌리 노드로 삽입한다.
 - 2) 최소 신장 트리에 삽입된 정점들과 이 정점들의 모든 인접 정점 사이에 있는 간선의 가중치를 조사한다. **간선 중에 가장 가중치가 작은 것을 골라 이 간선에 연결된 인접 정점을 최소 신장 트리에 삽입**한다. 단, 새로 삽입되는 정점은 최소 신장 트리에 삽입된 기존 노드와 사이클을 형성해서는 안 된다.
 - 3) 2)의 과정을 반복하다가 최소 신장 트리가 그래프의 모든 정점을 연결하게 되면 알고리즘 종료

02

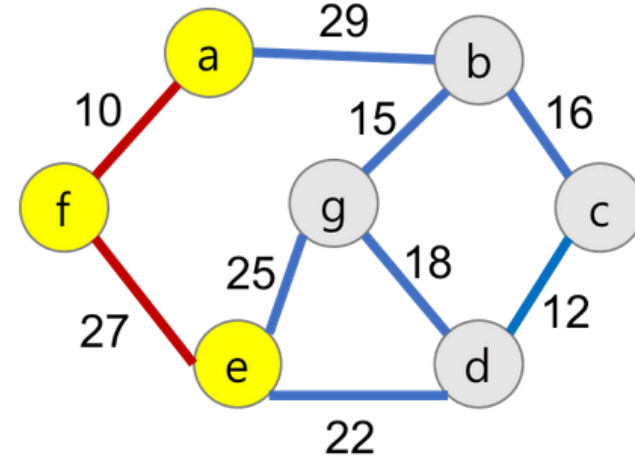
Prim 알고리즘

이것도 탐욕 알고리즘

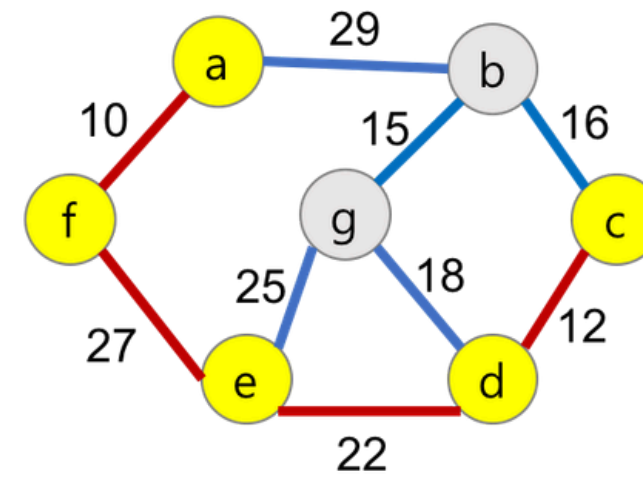
1) 시작 단계: 시작 정점만 포함



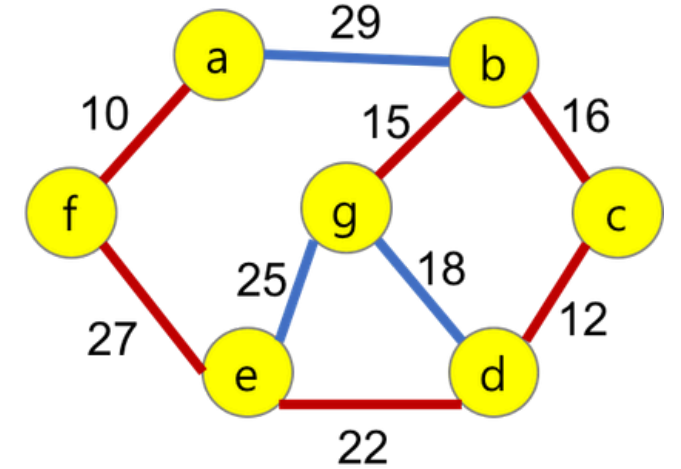
3)



5)

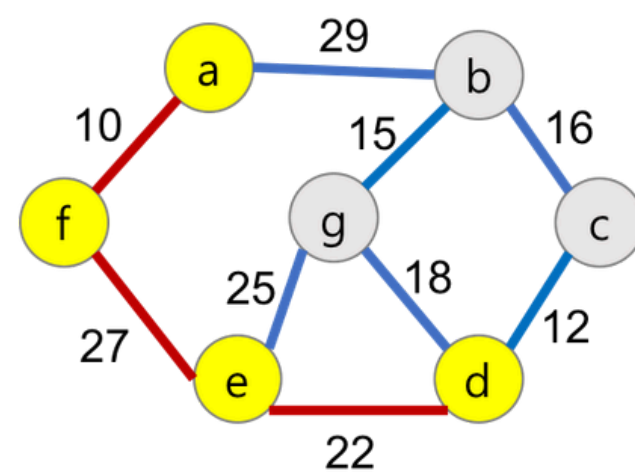
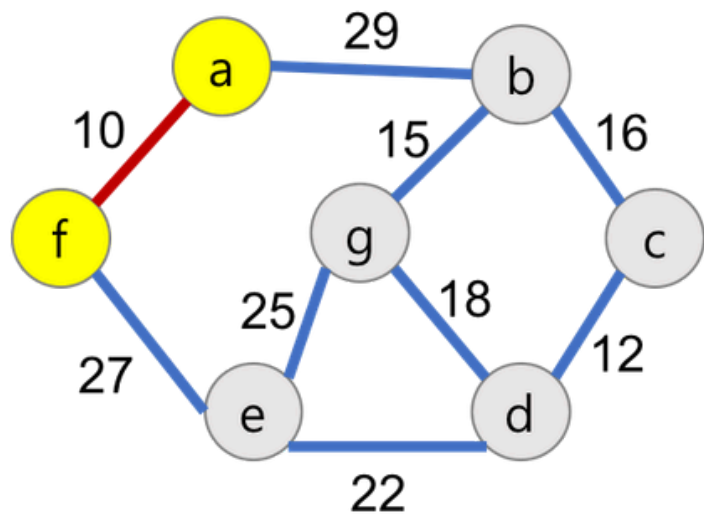


7) N-1개의 간선 생성. 종료

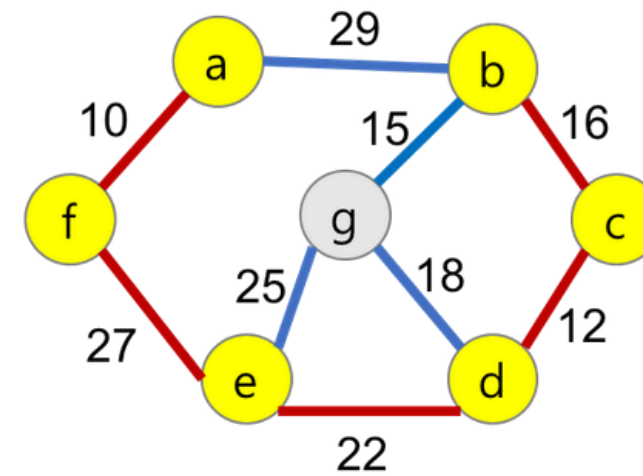


2) 인접 정점 중 최소 간선으로 연결된 정점 선택

4)



6)



02

Prim 알고리즘

Kruskal과 비교

	크루스칼	프림
접근 방식	간선 중심	정점 중심
시작점	없음 (전체 간선 대상)	임의의 정점 1개
동작 방식	가중치 낮은 간선부터 추가	현재 트리에서 가장 가까운 정점 추가
사이클 처리	Union-Find로 사이클 검사	방문 여부로 자동 방지
시간복잡도	$O(E \log E)$	$O(E \log V)$
유리한 그래프	희소 그래프 (간선 적을 때)	밀집 그래프 (간선 많을 때)

03

허프만 코딩

짧게 압축하기

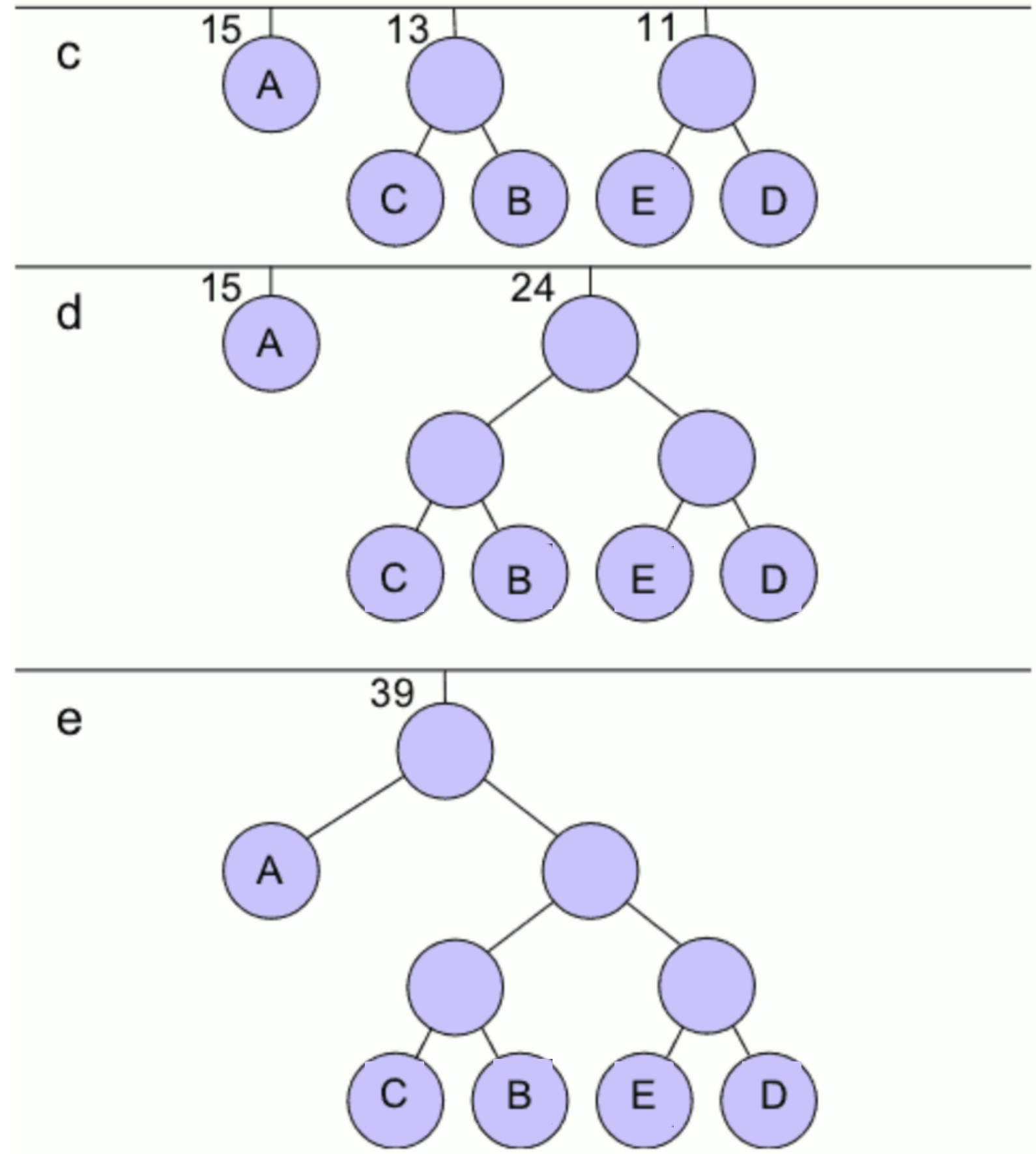
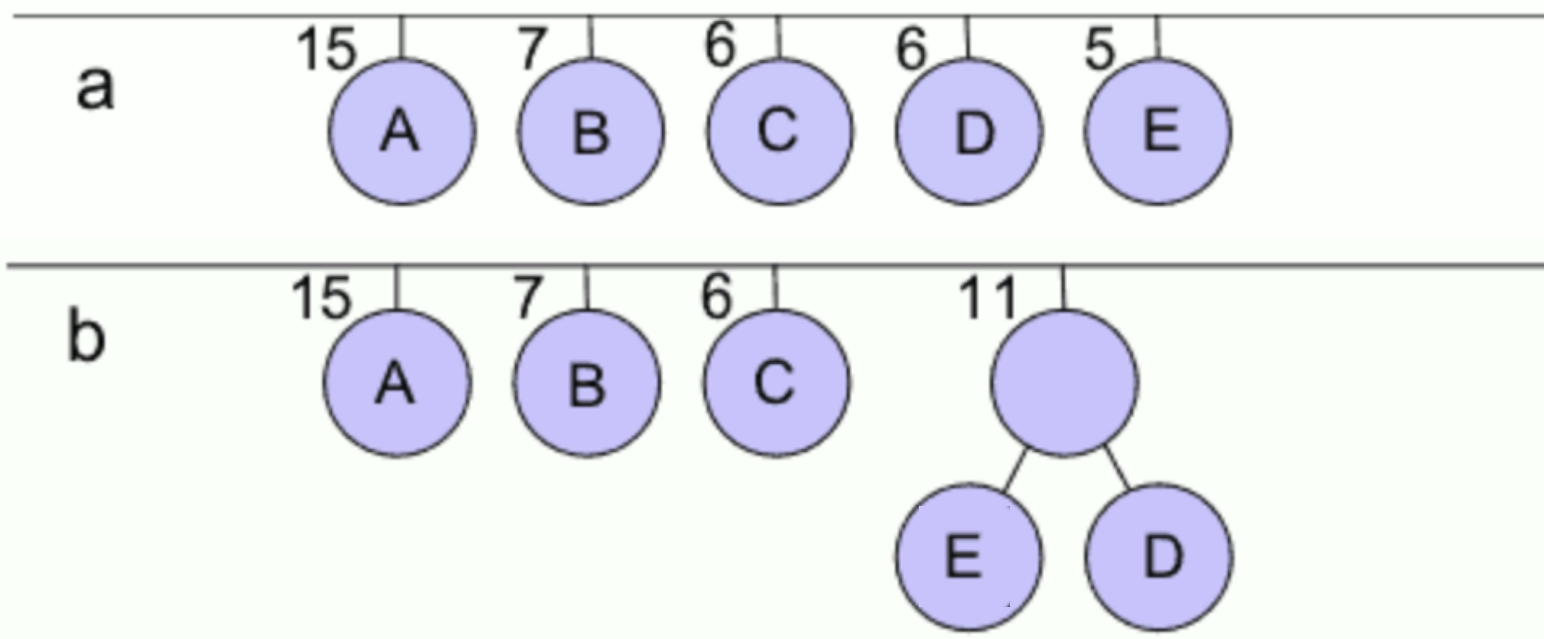
- 허프만 코딩(Huffman coding): 입력 파일의 문자 빈도 수를 바탕으로 파일을 압축한다. 자주 나오는 문자는 짧은 코드로 변환해야 짧게 압축된다는 것이 기본 원리. 파일 압축, 이미지 파일, 음악 파일 압축 등에 사용된다.
- 허프만 트리: 허프만 코딩에서 접두어 코드를 표현하기 위해 필요한 이진 트리
 - 리프 노드: 기호(문자)
 - 접두어 코드: 앞서 나온 문자가 다음에 나올 문자의 접두어가 되면 안 되는 가변 길이 코드
 - 고정 길이 코드: 모든 코드의 길이가 똑같은 값을 갖는 코드

03

허프만 코딩

허프만 트리 만들기

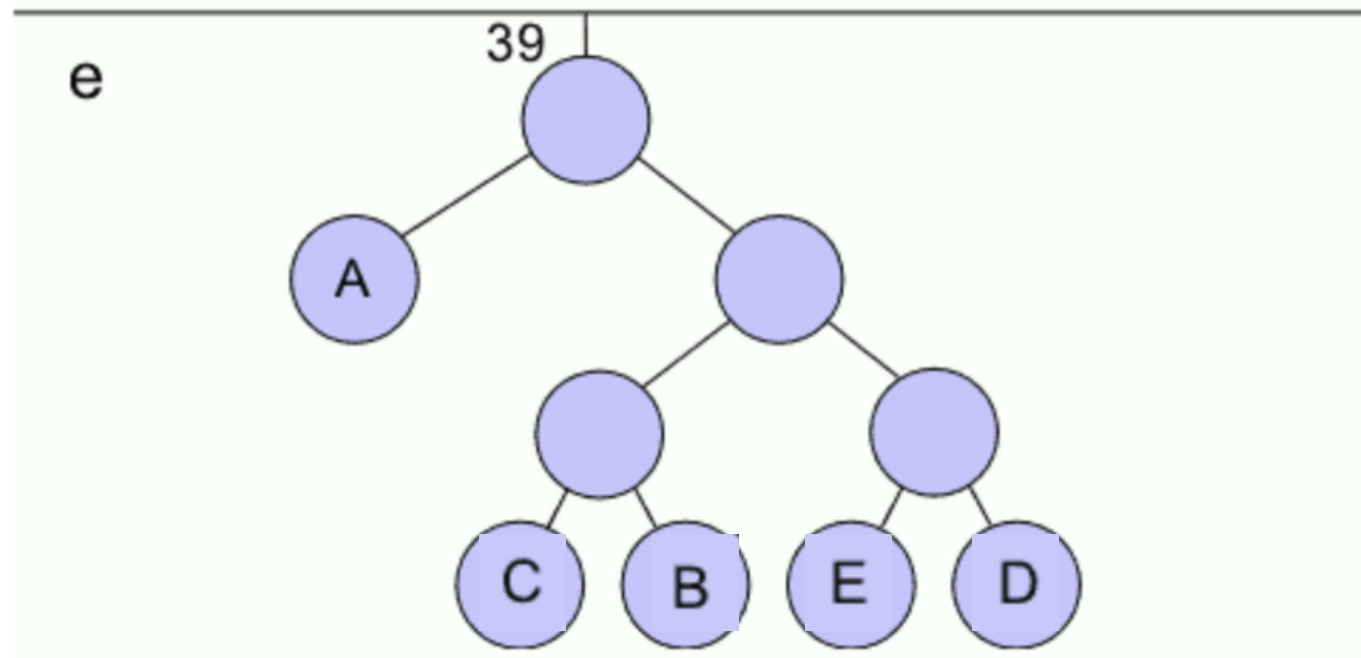
- 압축할 파일을 스캔하여 각 문자의 빈도 수를 계산한다.
- 빈도 수를 우선순위로 최소힙을 구성한다.
- 빈도 수가 가장 작은 두 노드를 삭제한다. (해 선택)
- 삭제한 두 노드 중 작은 것을 왼쪽 자식 노드, 큰 것을 오른쪽 자식 노드로 삽입한다. (실행 가능성 검사)
- 노드가 하나가 남을 때까지 반복한다.
- 마지막 노드가 루트 노드가 된다. (해 검사)



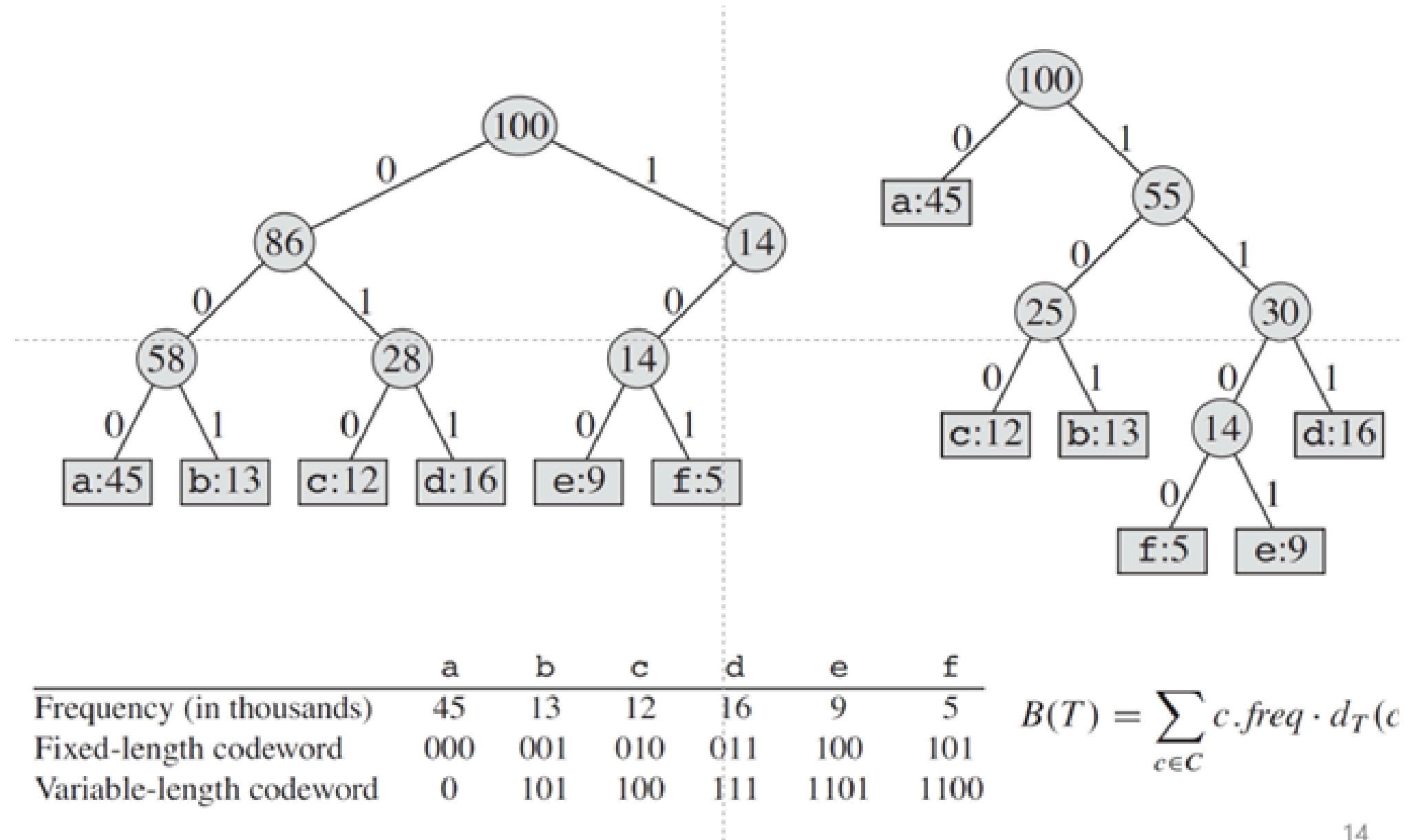
03

허프만 코딩

데이터 압축과 압축 해제



- A: 0
- B: 101
- C: 100
- D: 111
- E: 110



- 압축 해제 시에는 트리 순회를 반복한다.
- 매번 루트 노드에서 시작하고, 리프 노드를 만날 때까지 비트를 읽는다.
- 예) 001101111010110111000

04

탐구 문제 5번

```
1 def solution(number, k):
2     stack = []
3     for num in number:
4         while stack and stack[-1] < num and k > 0:
5             stack.pop()
6             k -= 1
7         stack.append(num)
8     print(stack)
9     print(k)
10    print()
11
12    if k > 0:
13        stack = stack[:-k]
14
15    return "".join(stack)
```

- 새로운 숫자보다 작은 숫자는 삭제
- 단, k 이내로
- 마지막 숫자까지 순회

Q&A
